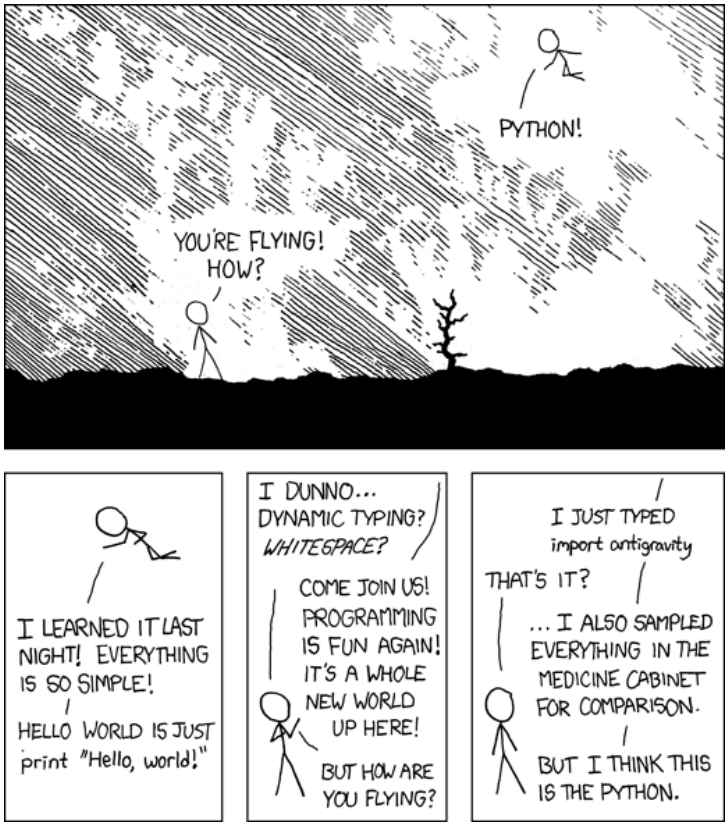
Reference and Practice Handout

Lecture-01: Assembly Arithmetic

Computer Organization & Assembly Language Programming
CS 2160 || Fall 2026
Instructor: Himon Thakur



Python

(Randall Munroe; CC BY-NC 2.5; https://xkcd.com/353/)

This document is a supplementary resource alongside course materials and class lectures. You are expected to go through and utilize it by yourself. You may read a section, then try solving the short exercises before moving on. Answers are provided at the end.

This handout will cover:

- What **RISC-V** is, and what **RV32I** means.
- How a C program becomes something the processor can run.
- The processor's **registers** and how assembly names them.
- Translating C arithmetic into RISC-V: `add`, `sub`, `addi`.
- **Multiplying** without a multiply instruction, using *shifts* and *adds*.

I used Claude Opus 5.5 to help me format the LaTeX code. All contents were produced, edited, or verified by me.

1. What Is RISC-V?

An **instruction set architecture (ISA)** is the vocabulary of commands a processor understands. Each **instruction** tells the processor to do **one operation** on **zero or more operands**. RISC-V is one such vocabulary, and it is the one we use in this course.

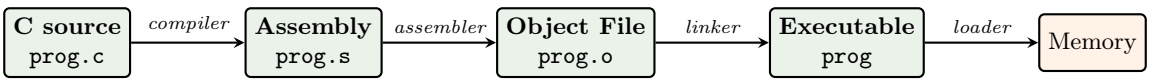
Style	Idea	Examples
CISC (complex instruction set)	Many instructions, some doing a lot of work each, with varying sizes.	x86 (most laptops and desktops)
RISC (reduced instruction set)	Fewer, simpler instructions of a fixed size; easier to build fast hardware for.	ARM (phones), MIPS, RISC-V

RISC-V was developed at UC Berkeley (it is the fifth RISC design from that group) and is an *open* standard: anyone can build a RISC-V processor without paying a license fee. It is used in embedded systems, microcontrollers, and AI/edge hardware.

RISC-V is a *family*: a small base set plus optional extensions.

Name	Meaning
RV32I	**R**ISC-**V**, **32**-bit registers, base **I**nteger instructions (**our focus**)
RV64I, RV128I	the same base with 64-bit or 128-bit registers
Extensions	letters added for more features, e.g., **M** (multiply/divide), **F**/**D** (floating point)

1.1. How a program is made



- The **compiler** translates C into assembly (human-readable instructions). This course is about this level.
- The **assembler** turns each assembly instruction into its 32-bit binary encoding.
- The **linker** joins object files and libraries into one executable.
- The **loader** (part of the operating system) copies the executable from disk into memory and starts it.

2. Registers

Registers are tiny, very fast storage locations inside the processor. RISC-V has **32 general-purpose registers**, each holding 32 bits. Arithmetic instructions work *only* on registers (and small constants), never directly on memory.

- Registers are numbered from 0. Assembly languages mark register numbers with a prefix: `x0`, `x1`, ... in RISC-V; `$0`, `$1`, ... in MIPS.
- Each also has a **mnemonic** (a nickname) that describes its usual job. Write `a0` or `x10`: they are the same register.
- The **program counter** (`pc`) holds the address of the current instruction. It is a special register, *not* one of the 32 general-purpose ones.

Mnemonic	Register	Meaning
`zero`	`x0`	always 0; writing to it has no effect
`ra`	`x1`	return address
`sp`	`x2`	stack pointer
`t0-t2`	`x5-x7`	temporaries
`s0-s1`	`x8-x9`	saved registers
`a0-a7`	`x10-x17`	function arguments; `a0` (and `a1`) also hold the return value
`s2-s11`	`x18-x27`	saved registers
`t3-t6`	`x28-x31`	temporaries

For now, the ones you need are `zero`, `t0`–`t6` (7 temporaries for scratch work), and `a0`–`a7`.

Three kinds of instructions:

- **Arithmetic/logic** instructions compute with register values.(**This lecture.**)
- **Load/store** instructions move data between registers and memory. **(Lecture 02)**
- **Conditional** (branch) instructions test register values to decide what runs next. (**Lecture 03**)

Exercise 1

(a) What value does `x0` always hold? (b) Which register holds a function's first argument? (c) Is `PC` one of the 32 general-purpose registers?

3. Addition and Subtraction

An arithmetic instruction has the shape `operation dest, src1, src2`
The **destination comes first**, then the inputs: just like `dest = src1 op src2` in C.

Example 1: `a = b + c;`

Before translating, we must decide *where* each variable lives. Assume `b` is in `a0`, `c` is in `a1`, and `a` goes in `t1`.

C	RISC-V
`a = b + c;`	`add t1, a0, a1`

`add` is the *operation*, `t1` the *output operand*, `a0` and `a1` the *input operands*.

Example 2: `d = e - f;`

Assume `e` in `t0`, `f` in `t5`, `d` goes in `a1`: `sub a1, t0, t5` (`a1` = `t0` − `t5`).
Order matters for subtraction: `sub a1, t5, t0` would compute `f - e`.

Example 3: Adding a constant: `addi`

When one input is a constant (an **immediate**), use `addi`: `x = y + 20;` → `addi t0, a0, 20`
- The immediate is a 12-bit signed number: −2048 to +2047.
- There is no `subi`. To subtract a constant, add a negative one: `x = y - 5;` → `addi t0, a0, -5`.

Example 4: Longer expression: `f = (g + h) - (i + j);`

RISC-V does one operation per instruction, so break the expression into steps and keep partial results in temporaries. With `g, h, i, j` in `a0`–`a3` and `f` in `t0`:

```
add  t1, a0, a1      # t1 = g + h
add  t2, a2, a3      # t2 = i + j
sub  t0, t1, t2      # f  = t1 - t2
```

Check: g=10, h=5, i=3, j=4 gives $15 - 7 = 8$.

Exercise 2

Translate `a = b + c - d;` with `b` in `a1`, `c` in `a2`, `d` in `a3`, and `a` in `a0`.

Exercise 3

Translate `x = y - 5;` with `y` in `a0` and `x` in `t0`.

4. Multiplying by a Power of 2

RV32I has **no multiply instruction** (it lives in the optional "M" extension). But multiplying by 2^k is easy: **shift the bits left by k places**. That is `slli` (shift left logical immediate).

Why it works. In decimal, multiplying by 10 appends a 0: $37 \times 10 = 370$. In binary, multiplying by 2 appends a 0, because every bit moves to a place worth twice as much. Doing that k times multiplies by 2^k.

Example 5: 6×8

$8 = 2^3$, so shift left by 3: $\quad 6 = \texttt{110}_2 \;\rightarrow\; \texttt{110000}_2 = 48$.

k	1	2	3	4	5	6	7	8	9	10
2^k	2	4	8	16	32	64	128	256	512	1024

Worth memorizing. Combine with prefixes for bigger values: $2^{10} \approx$ a thousand (Ki), $2^{20} \approx$ a million (Mi), $2^{30} \approx$ a billion (Gi).

Example 6: `g = h * 256;`

$256 = 2^8$, so shift by 8. With `h` in `t3` and `g` in `t4`:

```
slli t4, t3, 8
```

(Pseudo-code like "`mul g, h, 256`" is not real RISC-V. Even with the M extension, `mul` only takes registers, not a constant.)

Watch out

Bits that shift off the left end are lost. If the true product does not fit in 32 bits, the result is wrong (overflow), and RISC-V gives no warning. Example: `0x40000000` shifted left by 2 is `0x00000000`. Shifting does work for negative numbers in two's complement, as long as the result fits: $-5 \ll 4 = -80$.

Exercise 4

What is $13 \ll 2$? Give the answer in binary and decimal, and check that it equals 13×4.

5. Multiplying by Other Constants

Write the constant as a sum (or difference) of powers of 2, multiply by each with a shift, and combine with `add` or `sub`. Each 1-bit in the constant's binary form is one shift.

Example 7: `g = h * 18;`

$18 = 16 + 2 = 2^4 + 2^1$ (binary `10010`), so $h \times 18 = (h \ll 4) + (h \ll 1)$.
With `h` in `t3`, `g` in `t4`, and `t0`, `t1` free:

```
slli t0, t3, 4       # t0 = h * 16
slli t1, t3, 1       # t1 = h * 2
add  t4, t0, t1      # g  = h * 18
```

Using one fewer temporary: put the first partial result straight into the destination.

```
slli t4, t3, 4       # t4 = h * 16
slli t1, t3, 1       # t1 = h * 2
add  t4, t4, t1      # t4 = h * 18
```

Check: $h = 7$ gives $112 + 14 = 126 = 7 \times 18$.

Example 8: Using subtraction: `x * 7`

$7 = 4 + 2 + 1$ needs two shifts and two adds. But $7 = 8 - 1$:

```
slli t0, a0, 3        # t0 = x * 8
sub  a0, t0, a0       # a0 = x*8 - x = x * 7
```

Look for a nearby power of 2 when the constant has many 1-bits.

Exercise 5

Write RISC-V for `a0 = a0 * 10`. Use `t0` and `t1` as scratch.

6. Putting It Together

Instruction summary so far:

Instruction	Example	Meaning	Notes
add	`add x5, x6, x7`	x5 = x6 + x7	three registers
subtract	`sub x5, x6, x7`	x5 = x6 − x7	three registers
add immediate	`addi x5, x6, 20`	x5 = x6 +20	constant: −2048 to 2047
shift left	`sll x5, x6, x7`	x5 = x6 ≪ x7	amount from a register
shift left imm.	`slli x5, x6, 3`	x5 = x6 ≪ 3	multiply by 2^3

Every one has an immediate (`-i`) version **except** `sub`.

6.1. A whole function: `arith`

```
int arith(int x, int y, int z) {
    int t1 = x + y;
    int t2 = z + t1;
    int t3 = x;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 + t5;
    return rval;
}
```

Arguments arrive in `a0` (`x`), `a1` (`y`), `a2` (`z`); the result goes back in `a0`.

```
arith:
    add  t1, a0, a1       # t1 = x + y
    add  t2, a2, t1       # t2 = z + t1
    add  t4, a1, a1       # t4 = 2y
    add  t4, t4, a1       # t4 = 3y
    slli t4, t4, 4        # t4 = 3y * 16 = 48y
    add  t5, a0, t4       # t5 = x + 48y      (t3 = x needs no instruction)
    add  a0, t2, t5       # rval = t2 + t5, placed in a0 to return it
    ret                   # return to the caller
```

- `t3 = x` **costs nothing:** the compiler just uses `a0` wherever `t3` appears.
- $y \times 48$**:** $48 = 3 \times 16$, so compute $3y$ with two adds, then shift by 4.
- The C variable names `t1`, `t2`, ... happen to match the register names here, but they don't have to.

Exercise 6

Trace `arith` with `x` = 1, `y` = 2, `z` = 3. Write the value of each register after each instruction. What is returned?

6.2. Translate it yourself: `silly_math`

```
int silly_math(int a, int b) {
    int k = (b - a) + 11;
    return k * 42;
}
```

Plan. `a` is in `a0`, `b` in `a1`; return in `a0`; use `t0`–`t6` freely.

1. $k = (b - a) + 11$: one `sub`, one `addi`.
2. $42 = 32 + 8 + 2$. One neat way to group it: $42k = 2\big(4(4k + k) + k\big)$, which reuses partial results:

$$4k + k = 5k, \qquad 4 \cdot 5k + k = 21k, \qquad 2 \cdot 21k = 42k.$$

```
silly_math:
    sub  t0, a1, a0       # t0 = b - a
    addi t0, t0, 11       # t0 = k = (b - a) + 11
    slli t1, t0, 2        # t1 = 4k
    add  t1, t1, t0       # t1 = 5k
    slli t1, t1, 2        # t1 = 20k
    add  t1, t1, t0       # t1 = 21k
    slli a0, t1, 1        # a0 = 42k
    ret
```

Exercise 7

Run `silly_math` by hand with `a` $= 1$, `b` $= 2$. Then with `a` $= 10$, `b` $= 0$.

Recap

Remember these

1. RV32I = RISC-V, 32-bit registers, base integer instructions. RISC means simple, fixed-size instructions.
2. C → (compiler) → assembly → (assembler) → object → (linker) → executable → (loader) → memory.
3. 32 registers, `x0`–`x31`; `x0` is always 0. `a0`–`a7` carry arguments; `a0` returns the result; `t0`–`t6` are scratch.
4. Instruction shape: `op dest, src1, src2`. Destination first.
5. Before translating, decide which register holds each variable.
6. `addi` adds a constant; there is no `subi` (use a negative constant).
7. $x \times 2^k$ = `slli` by k. Other constants: split into powers of 2 and add (or subtract).
8. Shifting can overflow silently; bits off the left end are lost.

Answer key for Exercises

Ex. 1. (a) 0. (b) `a0` (`x10`). (c) No, `PC` is a separate special register.

Ex. 2. `add t0, a1, a2` then `sub a0, t0, a3`. (Check: $10 + 7 - 4 = 13$.)

Ex. 3. `addi t0, a0, -5`.

Ex. 4. $13 = 1101_2 \rightarrow 110100_2 = 52 = 13 \times 4$.

Ex. 5. $10 = 8 + 2$: `slli t0, a0, 3` `slli t1, a0, 1` `add a0, t0, t1`. (Check: $13 \rightarrow 130$.)

Ex. 6.

After	Register value
`add t1, a0, a1`	`t1` $= 1 + 2 = 3$
`add t2, a2, t1`	`t2` $= 3 + 3 = 6$
`add t4, a1, a1`	`t4` $= 4$
`add t4, t4, a1`	`t4` $= 6$
`slli t4, t4, 4`	`t4` $= 96$
`add t5, a0, t4`	`t5` $= 1 + 96 = 97$
`add a0, t2, t5`	`a0` $= 6 + 97 =$ **103**

Ex. 7. $a = 1, b = 2$: $k = 12$, returns $12 \times 42 =$ **504** (`t1`: 48, 60, 240, 252; `a0` = 504).
$a = 10, b = 0$: $k = 1$, returns **42**.

Student Practice Worksheet

Part 1: Concepts

Q1. Put these in order and name the tool between each pair: executable, assembly file, C source, object file, program in memory.

Q2. What does each part of the name **RV32I** stand for?

Part 2: Translate to RISC-V

Unless stated otherwise, `x` is in `a0`, `y` is in `a1`, and the result `z` goes in `a0`. Use `t0`–`t6` freely.

Q3. `z = x * 64;`

Q4. `z = x * 20;`

Q5. `z = x * 31;` (Try to use only two instructions.)

Q6. `int f(int x, int y) { return 3 * x + 12 * y; }`

Part 3: Think About It

Q7. `a0` = `0x40000000` (2^{30}). What is in `a0` after `slli a0, a0, 2`? Why isn't it 2^{32}?

Q8. Why does RISC-V have `addi` but no `subi`?

Step-by-Step Solutions & Explanations

Q1: Building a program Answer: C source $\xrightarrow{\text{compiler}}$ assembly file $\xrightarrow{\text{assembler}}$ object file $\xrightarrow{\text{linker}}$ executable $\xrightarrow{\text{loader}}$ program in memory.

Q2: RV32I Answer: RV = RISC-V; **32** = 32-bit registers; **I** = the base Integer instruction set.

Q3: $\times 64$

- $64 = 2^6 \rightarrow$ `slli a0, a0, 6`.

Q4: $\times 20$

- $20 = 16 + 4$:

```
slli t0, a0, 4       # x * 16
slli t1, a0, 2       # x * 4
add  a0, t0, t1      # x * 20
```

Check: $13 \rightarrow 260$.

Q5: $\times 31$

- $31 = 32 - 1$ (as a sum it would need five 1-bits: $16 + 8 + 4 + 2 + 1$).

```
slli t0, a0, 5       # x * 32
sub  a0, t0, a0      # x * 32 - x = x * 31
```

Check: $13 \rightarrow 403$.

Q6: $3x + 12y$

- $3x = 2x + x$; $\quad 12y = 8y + 4y$.

```
f:
    slli t0, a0, 1       # t0 = 2x
    add  t0, t0, a0      # t0 = 3x
    slli t1, a1, 3       # t1 = 8y
    slli t2, a1, 2       # t2 = 4y
    add  t1, t1, t2      # t1 = 12y
    add  a0, t0, t1      # return 3x + 12y
    ret
```

Check: $x = 5, y = 3$ gives $15 + 36 = 51$. (Another option: $12y = 3y \ll 2$.)

Q7: Overflow Answer: `a0` becomes `0x00000000`. The only 1-bit was bit 30; shifting it 2 places moves it to bit 32, which does not exist in a 32-bit register, so it is lost. The true answer 2^{32} needs 33 bits.

Q8: No `subi` Answer: It isn't needed. The immediate is a signed number, so `addi rd, rs, -5` already subtracts 5. Leaving out `subi` keeps the instruction set smaller and simpler (the RISC idea).